Week 2, Day 1 — presenter notes

Slides: open `week-02-day-01.html` in Chrome/Firefox → **F** fullscreen → arrows. **N** shows the speaker note for the current slide. **P** prints to PDF.

This session is **types and values**. No slices. No `if` / `for`. No SMS code. First `go test`.

Timing (3–3.5 hours)

Clock	Block	Slides
0:00–0:15	Homework check, leftover installs	1–2
0:15–0:25	Outcomes + why types are SE	3–4
0:25–0:50	`var`, `:=`, `=`, `const`, iota (light)	5–8
0:50–1:20	Types, zero values, conversions (**live** `zeros.go`)	9–12
1:20–1:40	Operators (skip bitwise live if late)	13–15
1:40–2:20	`fmt`, `strings`, runes vs bytes (**live** UTF-8)	16–19
2:20–2:40	`time`, naming, CLI + `go test` (**live**)	20–23
2:40–end	Lab: worksheet + greet + green test	24–end

There is **no agenda slide** and **no homework slide**. Homework stays on the student handout. If conversion + UTF-8 eat time, cut bitwise, never cut zero values or `go test`.

Live demo script

1. Zero values (`~/epic-go/week02/zeros.go`)

```
package main

import "fmt"

func main() {
    var seats int
    var label string
    var open bool
    fmt.Printf("zeroes: %d %q %t\n", seats, label, open)
    celsius := 33
```

```
    fmt.Println(float64(celsius)*9/5 + 32)
}
```

Show `celsius + 0.5` failing. Then `float64(celsius)`.

```
gofmt -w zeros.go
go run zeros.go
```

2. UTF-8 (`~/epic-go/week02/runes.go`)

Type `සිංහල` and `café`. Print `len` next to `utf8.RuneCountInString` (15 bytes / 5 runes, not "6 letters"). The vowel sign ් is its own code point. `range` over the string once so they see the index jump. Do **not** treat `word[0]` as a letter.

3. CLI (`~/epic-go/week02/greet.go`)

```
first := flag.String("first", "", "first name")
last := flag.String("last", "", "last name")
flag.Parse()
fmt.Println(*first, *last)
```

`go run greet.go -first Ada -last Lovelace`. The `*` is "read the box". Pointer theory is week 8.

4. First test (`~/epic-go/week02/names/`)

```
mkdir -p ~/epic-go/week02/names
cd ~/epic-go/week02/names
go mod init epiclearn/week02
```

`go mod init` is a **ritual**, not a lecture. Meaning is week 4.

Fail `FullName` first (return only `first`). Then:

```
go test
```

Red → green. Students must see both.

Lab gate (they do not leave without)

- [ ] Worksheet cells filled (zeros + which conversions compile)
- [ ] `zeros.go` runs

- [ ] `greet -first … -last …` prints one name
- [ ] `go test` is green for `FullName("Ada", "Lovelace")`
- [ ] GitHub URL collected

Homework

Ten exercises on the handout + improve `FullName` with `TrimSpace` and a second test.

Typical failures

Symptom	Fix
`:=` at package level	`var` at the top of the file
`no new variables on left side of :=`	use `=`
`invalid operation: int + float64`	explicit conversion
`go test`: no `go.mod`	`go mod init epiclearn/week02`
`undefined: flag`	`import "flag"`
prints a pointer like `0xc000…`	they forgot `*`
`len("café")` is 5 and they argue	bytes vs runes slide
`a++` inside `Println`	`++` is a statement

What not to teach today

`if` / `for` / `switch`, slices as a topic, maps, methods, pointers as theory, modules as theory, REST, the SMS repo. Tease week 3 only.